API SECURITY RISK ANALYSIS REPORT

Prepared By: Tshepo Eugene Letsoalo

Internship ID: FIT/MAR26/CS7439

Role: Cyber Security Intern

Date: April 13, 2026

1. Executive Summary

This report details a security risk analysis of the **JSONPlaceholder REST API** (https://jsonplaceholder.typicode.com). The audit was conducted using **Postman** and **Postbot AI** to identify vulnerabilities based on the **OWASP API Security Top 10**. The primary goal was to assess data exposure risks and server configuration weaknesses that could affect a modern SaaS platform.

2. Technical Methodology

To perform this audit, the following professional tools and steps were utilized:

- **Tooling:** Postman Desktop Agent was used to intercept and analyze HTTP Request/Response cycles.
- **Endpoint Tested:** GET /users/1
- **Analysis Focus:** Payload inspection for PII (Personally Identifiable Information) and Response Header audit for security hardening.

3. Identified Security Risks

Finding A: Excessive Data Exposure (OWASP API3:2023)

- **Observation:** Upon sending a request to the /users/1 endpoint, the API returns a comprehensive JSON object. This includes sensitive data points such as the user's full physical address and precise geographic coordinates (lat, lng).
- **Severity: MEDIUM / HIGH**
- **Business Impact:** In a production SaaS environment, returning unnecessary sensitive data increases the attack surface. If intercepted, this data leads to privacy violations (GDPR/POPIA) and potential physical security risks for users.
- **Remediation:** Implement **Response Filtering**. The backend should be configured to return only the fields requested by the client-side application (e.g., just the name and email) rather than the entire database record.

Finding B: Security Misconfiguration – Missing Headers (OWASP API7:2023)

- **Observation:** Inspection of the Response Headers reveals the absence of critical security-hardening headers, specifically Strict-Transport-Security (HSTS) and Content-Security-Policy (CSP).
- **Severity: MEDIUM**
- **Business Impact:** The lack of HSTS allows for Protocol Downgrade attacks (forcing a user from https to http), while the lack of CSP increases the risk of Cross-Site Scripting (XSS) and data injection.

- **Remediation:** Enable security headers at the server level (Nginx/Apache) or via application middleware (e.g., helmet for Node.js). Ensure Strict-Transport-Security is set with a long max-age.

4. Risk Classification Table

Risk ID	Vulnerability	Severity	Status
API-01	Excessive Data Exposure	Medium/High	Identified
API-02	Missing Security Headers	Medium	Identified
API-03	Lack of Rate Limiting	Low/Medium	Potential

5. Conclusion

The analysis confirms that while the API is functional, it follows a "trust-all" data model that exposes more information than necessary. By implementing the suggested remediations specifically data filtering and header hardening the organization can significantly reduce its risk profile and ensure compliance with modern security standards.

6. Evidence Artifacts

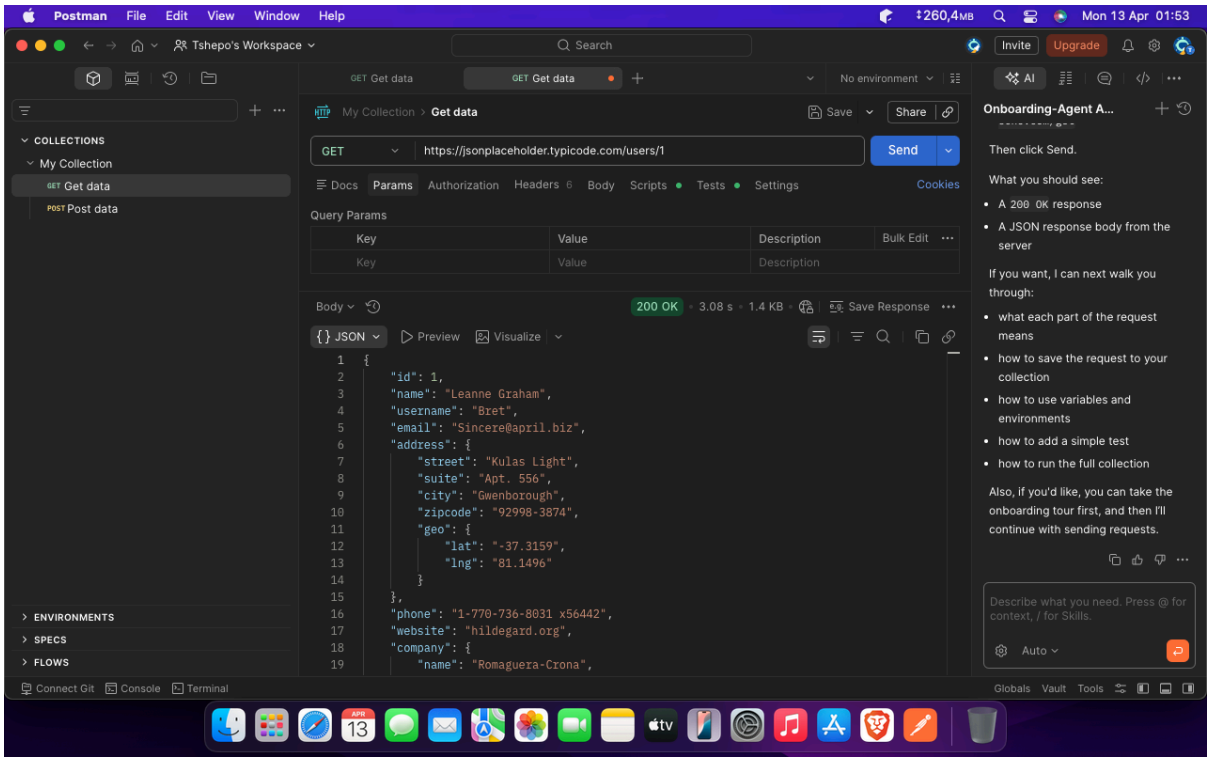

Figure 1: API Response Payload Analysis. > *This screenshot captures the JSON response from the /users/1 endpoint. The highlighted sections demonstrate* ***Excessive Data Exposure****, as the API unnecessarily returns the user's precise latitude and longitude (geo data) and full physical address in a single unauthenticated request.*

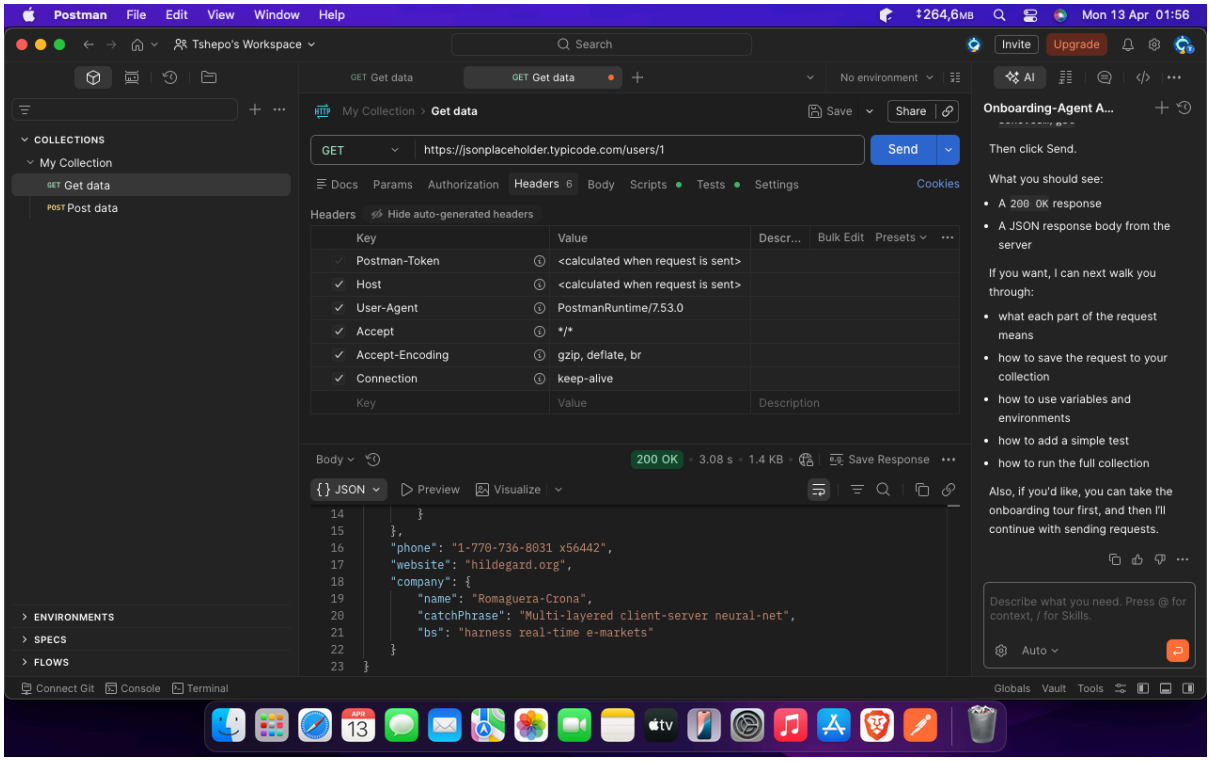

Figure 2: Response Header Security Audit. > *This screenshot displays the metadata returned by the server. The analysis confirms a* ***Security Misconfiguration****, specifically the absence of the Strict-Transport-Security (HSTS) and Content-Security-Policy (CSP) headers, leaving the connection vulnerable to protocol downgrade and injection attacks.*

7. Final Recommendation & Sign-off

Based on this analysis, it is recommended that the development team implements automated security header checks and response filtering as part of the CI/CD pipeline. These measures will ensure that PII is protected and that the API remains resilient against common web vulnerabilities.

Report End.